Assignment-9

* Problem Statement

Queues are frequently used in computer programming, and a typical example is the creation of a job queue by operating system, if the operating system does not use priorities then jobs are processed in order they enter system. write C program for simulating job queue. write function to add job and delete a job.

* objectives

1. Understand the Concept of queue and fifo
2. Implement a linear queue using array
3. Implement functions to add and delete jobs.
4. Simulate processing of jobs in order they enter the system.
5. Analyze the time & space complexity of queue operations.

* Test condition.

Test case	Input	Expected output
TC 01	Add one job	Job added successfully
TC 02	Add multiple jobs	Jobs stored in FIFO order
TC 03	Delete job	First job entered job is deleted.
TC 04	Delete from empty queue	Queue underflow message.

Test case	Input	Expected output
Test	Add job when queue is full	Queue overflow message
Test	Display pending jobs	All jobs displayed from front to rear

* Pseudo code:

```
START
Initialize front = -1, rear = -1
REPEAT
  Display menu
  Accept choice
  IF choice = 1
    IF rear = MAX - 1
    Display "overflow"
  ELSE
    Accept job
    If front = -1, Set front = 1
    Increment rear
    Add job at rear
  END IF
ELSE IF choice = 2
  IF front = -1
    Display "underflow"
  ELSE
    process and delete job from front
    Increment front
  END IF
ELSE IF choice = 3
    Display jobs from front to rear
ELSE IF choice = 4
  EXIT
UNTIL choice = 4
STOP
```

* Time Complexity

Operation	Time complexity
Add job (enqueue)	O(1)
Delete job (Dequeue)	O(1)
Display jobs	O(n)

* Conclusion:
The job queue simulation demonstrate the FIFO principle of linear queue jobs are added at rear and processed / deleted from the front in same order in which they enter the system.

* FAQ's

1. What is queue?
→ A queue is linear data structure in which elements are inserted from the rear and deleted from the front.

2. what is FIFO?
→ FIFO means First In, First out. The element inserted first is removed first.

3. Why is queue suitable for an operating system job queue?

⇒ It is suitable because it processes jobs in the same order in which they arrive, ensuring fairness.

4. What is queue overflow and queue underflow?

⇒

- Overflow

~~Trying~~ when we try to insert an element into a queue but the queue is full it is overflow.

- Underflow

when we try to delete an element from a queue but the queue is empty it is underflow.

5. What are applications of queue?

⇒

1. CPU scheduling
2. Printer Scheduling
3. Job scheduling
4. network data handling
5. Customer service systems.

6. What is circular queue?

⇒ A circular queue in which the last position is connected to

3. Why is queue suitable for an operating system job queue?
→ It is suitable because it processes jobs in the same order in which they arrive, ensuring fairness.

4. What is queue overflow and queue underflow?
→
- Overflow
  ~~Trying~~ when we try to insert an element into a queue but the queue is full it is overflow.
- Underflow
  when we try to delete an element from a queue but the queue is empty it is underflow.

5. What are applications of queue?
→
1. CPU scheduling
2. Printer scheduling
3. Job scheduling
4. network data handling
5. Customer service systems.

6. What is circular queue?
→ A circular queue in which the last position is connected to

the first position.

7. Why is circular queue preferred over a linear queue.

→ Circular queue reuses the empty spaces created after deletion, making better use of memory. better memory optimization.

8. What principle does circular queue follow.

→ It follows the FIFO (First In, First out) principle.

9. What are the basic operations performed on circular queue?

→ Enqueue
Dequeue
Display.

10. What is an overflow & underflow condition in circular queue?

→

- Overflow: Queue is full and insertion is attempted.
- Underflow: Queue is empty and deletion is attempted.

Experiment 09: Job Queue Simulation Using Linear Queue

C Source Code

```
#include <stdio.h>
#include <string.h>

#define MAX 5
#define NAME_LEN 50

typedef struct {
    int id;
    char name[NAME_LEN];
} Job;

Job queue[MAX];
int front = -1, rear = -1;

void addJob(void) {
    Job job;
    if (rear == MAX - 1) {
        printf("\nQueue Overflow! Cannot add more jobs.\n");
        return;
    }

    printf("Enter Job ID: ");
    if (scanf("%d", &job.id) != 1) {
        printf("Invalid Job ID.\n");
        while (getchar() != '\n') {}
        return;
    }
    printf("Enter Job Name (one word): ");
    scanf("%49s", job.name);

    if (front == -1)
        front = 0;
    queue[++rear] = job;
    printf("Job added successfully.\n");
}

void deleteJob(void) {
    if (front == -1 || front > rear) {
        printf("\nQueue Underflow! No jobs to process.\n");
        return;
    }

    printf("\nProcessing Job: ID=%d, Name=%s\n",
           queue[front].id, queue[front].name);
    front++;

    if (front > rear)
        front = rear = -1;
}

void displayJobs(void) {
    int i;
    if (front == -1) {
        printf("\nNo pending jobs in the queue.\n");
        return;
    }

    printf("\nPending jobs (front to rear):\n");
    for (i = front; i <= rear; i++)
        printf("ID=%d, Name=%s\n", queue[i].id, queue[i].name);
}

int main(void) {
    int choice;
    do {
        printf("\n--- JOB QUEUE SIMULATION ---\n");
        printf("1. Add Job\n2. Process/Delete Job\n3. Display Jobs\n4. Exit\n");
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1) {
            printf("Invalid choice.\n");
            return 1;
        }

        switch (choice) {
            case 1: addJob(); break;
            case 2: deleteJob(); break;
            case 3: displayJobs(); break;
            case 4: printf("Exiting program.\n"); break;
            default: printf("Invalid choice. Try again.\n");
        }
    } while (choice != 4);

    return 0;
}
```

Sample Input / Output Screenshot

Terminal-style screenshot generated from a compiled test run of the program.

